

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA0403

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Debugging or Optimization task

Code of Conduct:

I, **Ashira Bridget Susan S (192524316)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student

Ashira Bridget Susan S

192524316

Q1. Deadlock Debugging (Code Trace)

You are given the following pseudo-code for two processes sharing resources R1 and R2:

Process P1:

```
lock(R1)
wait(100 ms)
lock(R2)
critical section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R2)
wait(100 ms)
lock(R1)
critical section
unlock(R1)
unlock(R2)
```

Identify the exact condition that causes deadlock. Suggest an optimisation to eliminate the deadlock without changing process logic.

SOLUTION:

1. Cause of Deadlock

- P1 locks R1 and waits for R2.
- P2 locks R2 and waits for R1.
- Both processes hold one resource and wait indefinitely for the other.

Deadlock Conditions Present:

- Mutual Exclusion
- Hold and Wait
- No Preemption

- Circular Wait (Main cause)

2. Optimization

- Enforce resource ordering.
- Both processes must acquire resources in the same order.

Process P1:

```
lock(R1)
lock(R2)
Critical Section
unlock(R2)
unlock(R1)
```

Process P2:

```
lock(R1)
lock(R2)
Critical Section
unlock(R2)
unlock(R1)
```

Answer: Eliminating circular wait removes the deadlock without changing program functionality.

Q2. CPU Utilisation Problem

A system shows low CPU utilisation (30%) even though processes are ready. You analyse the scheduler log and find frequent transitions:

ready → waiting → ready → waiting → ready ... Debug the cause using process management concepts and propose a system-level optimisation to improve utilisation.

SOLUTION:

Cause

The processes are repeatedly changing states:

Ready → Waiting → Ready → Waiting

This indicates:

- Excessive I/O operations.
- Processes are spending most of their time waiting.
- CPU remains idle despite ready processes.

Why CPU Utilization is Low?

- Processes quickly request I/O.
- Scheduler frequently performs context switching.
- CPU cannot execute processes continuously.

Optimization

1. Use asynchronous/non-blocking I/O.
2. Increase multiprogramming level or improve CPU scheduling.

Examples:

- Multilevel Feedback Queue.
- Better load balancing.
- Overlap CPU and I/O operations.

Answer: Low utilization occurs because processes are I/O-bound. Optimizing I/O handling and scheduling improves CPU utilization.

Q3. Memory Bottleneck Debugging

A multi-threaded program frequently encounters page faults, slowing performance. The memory

map shows:

- Code: 5 MB
- Heap: 150 MB
- Stack: 1 MB per thread
- 25 threads running
- Physical RAM available: 2 GB

Identify the memory issue causing performance degradation and propose two optimisations. (10 Marks)

SOLUTION:

Given

Code = 5 MB

Heap = 150 MB

Stack = 1 MB/thread

25 Threads

Total Stack = 25 MB

RAM = 2 GB

Cause

Although RAM is sufficient:

- Multi-threading increases memory consumption.
- Large heap allocations.
- Frequent page faults indicate poor memory locality.

This causes:

- Excessive swapping.
- Cache misses.
- Page replacement overhead.

Optimizations

1. Reduce Memory Usage

- Optimize heap allocation.
- Use memory pools.
- Reduce unnecessary data structures.

2. Reduce Thread Overhead

- Use thread pools.
- Reduce stack size if possible.
- Limit active threads.

Answer: Performance degradation is caused by frequent page faults and inefficient memory usage.

Q4. Starvation Debugging

You observe that process P5 never gets CPU time in a multilevel queue scheduler:

- Q1 (Highest Priority): Interactive
- Q2: System
- Q3: Batch
- Time Quantum: Q1 = 5ms, Q2 = 10ms, Q3 = FCFS

P5 is in Q3 but keeps getting postponed. Debug the root cause of starvation and suggest an optimisation to ensure fairness. (10 Marks)

SOLUTION:

Cause

Q1 → Highest Priority

Q2 → Medium Priority

Q3 → Lowest Priority (P5)

Whenever:

Q1 receives processes

↓

CPU assigned to Q1

↓

Q2 receives processes

↓

CPU assigned to Q2

↓

P5 keeps waiting forever

P5 suffers starvation.

Optimization

Aging Technique

Increase process priority over time.

Example:

After waiting:

Q3 → promoted to Q2

Further waiting:

Q2 → promoted to Q1

Other optimization:

- Allocate fixed CPU share to lower queues.

Answer: Aging guarantees fairness and prevents starvation.

Q5. Context-Switch Overhead Debugging

A server performs 20,000 context switches/sec, causing high overhead. Log details show:

- Average burst time per thread: 2ms
- I/O wait frequency: very high
- Preemptive scheduler with quantum = 1ms

Explain why context switching is so high and propose two optimisations to reduce overhead. (10 Marks)

SOLUTION:

Given

20,000 Context Switches/sec

Burst Time = 2 ms

Quantum = 1 ms

High I/O frequency

Cause

Since:

Burst = 2 ms

Quantum = 1 ms

Each thread requires:

1 ms

↓

Preempted

↓

Context Switch

↓

Another 1 ms

↓

Context Switch

High I/O further increases switching.

Optimizations

Increase Time Quantum

Example:

1 ms $\rightarrow$ 4 ms

reduces:

- Preemption
- Scheduling overhead

Use Thread Pools

Avoid:

- Frequent thread creation.
- Frequent termination.

Answer: Small quantum size and excessive I/O are causing high context-switch overhead.

Q6. Semaphore Synchronization Debugging

A producer-consumer application occasionally hangs during execution. The semaphore values

observed are:

mutex = 1

empty = 0

full = 10

Buffer size = 10

Producer Code:

wait(mutex)

wait(empty)

produce item

signal(full)

signal(mutex)

Consumer Code:

wait(mutex)

wait(full)

consume item

signal(empty)

signal(mutex)

Identify the synchronization bug causing the system to hang. Explain which deadlock condition is being violated. Suggest an optimization to ensure correct synchronization without changing the producer- consumer functionality. (10 Marks)

SOLUTION:

Given

mutex =1

empty =0

full =10

Producer:

wait(mutex)

wait(empty)

produce

signal(full)

signal(mutex)

Problem

If:

empty = 0

Producer does:

wait(mutex)

↓

mutex becomes 0

↓

wait(empty)

↓

Blocked

Consumer:

wait(mutex)

↓

Cannot proceed

↓

Blocked

Both processes wait forever.

Deadlock Condition

The violated condition is:

Hold and Wait

Producer holds:

mutex

while waiting for:

empty

Optimization

Correct ordering:

Producer:

wait(empty)

wait(mutex)

produce

signal(mutex)

signal(full)

Consumer:

wait(full)

wait(mutex)

consume

signal(mutex)

signal(empty)

Answer: Acquiring counting semaphores before mutex prevents deadlock.

Q7. Thrashing Detection and Optimization

A virtual memory system reports the following statistics:

Number of processes: 50

Average page fault rate: 250 faults/sec

CPU Utilization: 18%

Disk Utilization: 95%

Free Frames Available: 2%

The operating system suspects thrashing. Debug the root cause of the performance degradation. Explain the relationship between page faults and CPU utilization. Propose two operating system optimizations to reduce thrashing. (10 Marks)

SOLUTION:

Given

Processes =50

Page Fault Rate =250/sec

CPU =18%

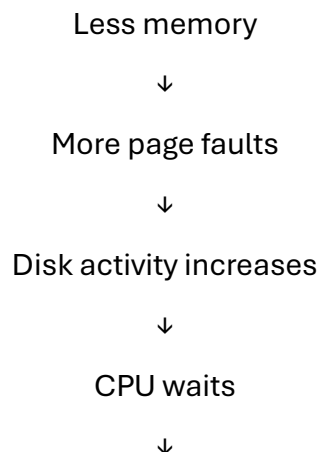
Disk =95%

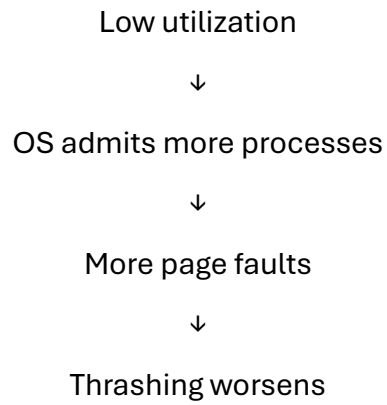
Free Frames =2%

Cause

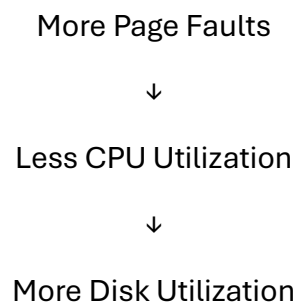
This is a classic case of thrashing.

Cycle





Relation



Optimizations

1. Reduce Degree of Multiprogramming.
2. Use Working Set Model or Page Fault Frequency control.

Other solutions:

- Increase RAM.
- Better page replacement algorithms.

Answer: Excessive page swapping causes low CPU utilization and heavy disk activity.

Q8. File System Performance Debugging

A database server experiences slow file access times. System logs indicate:

Average file size: 500 MB

Disk seeks per request: 120

Disk utilization: 92%

Cache hit ratio: 15%

Identify the file system bottleneck affecting performance. Explain why disk access latency is increasing. Suggest two optimizations to improve file access efficiency. (10 Marks)

SOLUTION:

Given

File Size =500 MB

Disk Seeks =120

Disk Utilization =92%

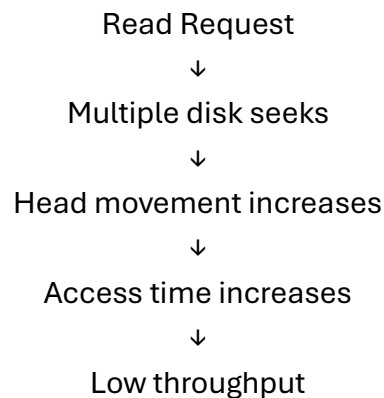
Cache Hit =15%

Cause

Problems:

- Excessive disk seeks.
- Poor caching.
- Fragmented file access.

Why Latency Increases?



Optimizations

Improve Caching

- Increase cache size.
- Use buffer cache efficiently.

Reduce Fragmentation

- Use contiguous allocation.
- Defragment storage periodically.

Answer: Excessive disk seeking and poor cache utilization cause slow file access.

Q9. Thread Scheduling Debugging

A web server creates one thread per client request. During peak hours:

Active Threads: 5,000

CPU Utilization: 60%

Memory Usage: 92%

Average Response Time: 12 seconds

Scheduler logs show frequent thread creation and termination. Identify the primary issue affecting server performance. Explain how thread management contributes to the delay. Recommend two optimizations to improve scalability and response time. (10 Marks)

SOLUTION:

Given

5000 Threads

CPU =60%

Memory =92%

Response Time =12 sec

Cause

The server uses:

1 Thread

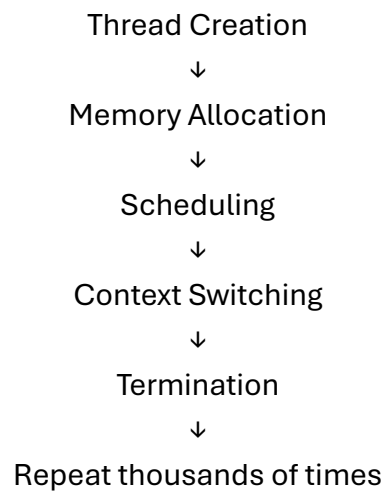
↓

1 Client

Problems:

- Frequent thread creation.
- Frequent termination.
- High memory usage.
- Scheduling overhead.

Delay Mechanism



Optimizations

Use Thread Pooling

Benefits:

- Reuse existing threads.
- Lower overhead.

Use Asynchronous Processing

Examples:

- Event-driven model.
- Non-blocking I/O.

Answer: Excessive thread management overhead is degrading scalability.

Q10. Disk Scheduling Optimization Problem

A disk scheduling log shows the following request queue:

98, 183, 37, 122, 14, 124, 65, 67

Current Head Position: 53

The system uses FCFS scheduling and reports:

Average seek time: Very High

Throughput: Low

User complaints regarding slow responseDebug why FCFS is causing poor performance in this

scenario. Compare FCFS with one suitable disk scheduling algorithm for this workload. Propose an optimization to improve seek time and overall throughput. (10 Marks)

SOLUTION:

Given

Queue:

098

183

037

122

014

124

065

067

Head =53

FCFS Scheduling

Why FCFS Performs Poorly?

Movement:

53→98

98→183

183→37

37→122

122→14

14→124

124→65

65→67

The disk head moves randomly, producing:

- High seek time.
- Low throughput.
- Slow response.

Better Algorithm

SSTF (Shortest Seek Time First)

From 53:

53→65

65→67

67→37

37→14

14→98

98→122

122→124

124→183

Advantages:

- Lower head movement.
- Better throughput.
- Faster response.

Optimization

Use:

- SSTF or SCAN scheduling.
- Request buffering and disk caching.

FCFS	SSTF
High seek time	Low seek time
Lower throughput	Higher throughput
Poor response time	Better response time
No optimization	Selects nearest request

Answer: FCFS causes excessive head movement. SSTF significantly reduces seek time and improves throughput.